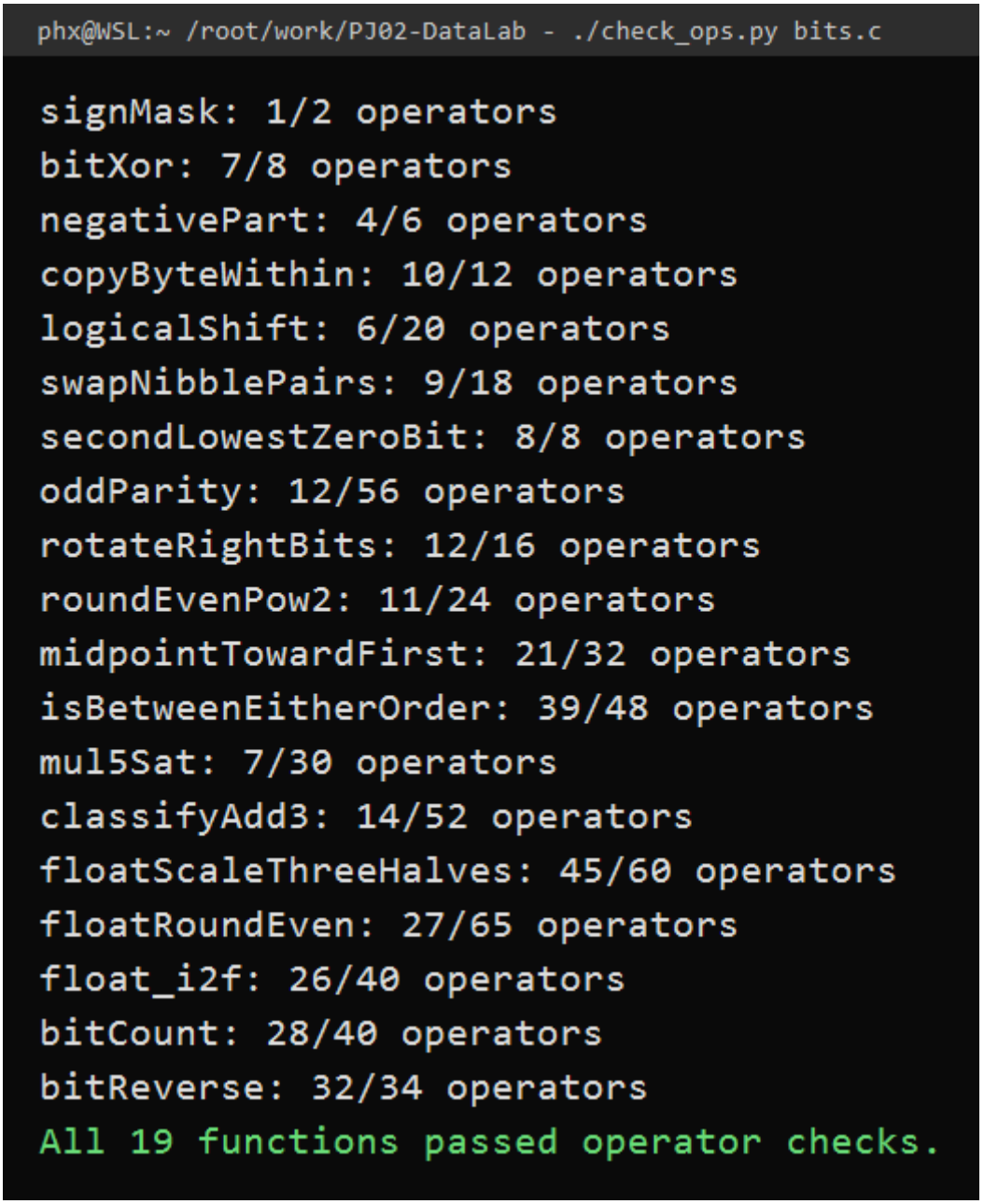
Lab 1 — Data Lab

> 说明：源仓库文件为 Windows CRLF 换行，直接 `./check_ops.py` 变成 `python3\r` 报错。测试在 Linux 侧工作副本中进行，并已用 `sed -i 's/\r$//'` 统一为 LF

一、运算符与操作数检查截图

下图是在项目目录中执行 `./check_ops.py bits.c` 的输出。每一行的「当前操作数/最大操作数」均满足限制，最后一行输出 `All 19 functions passed operator checks.`。



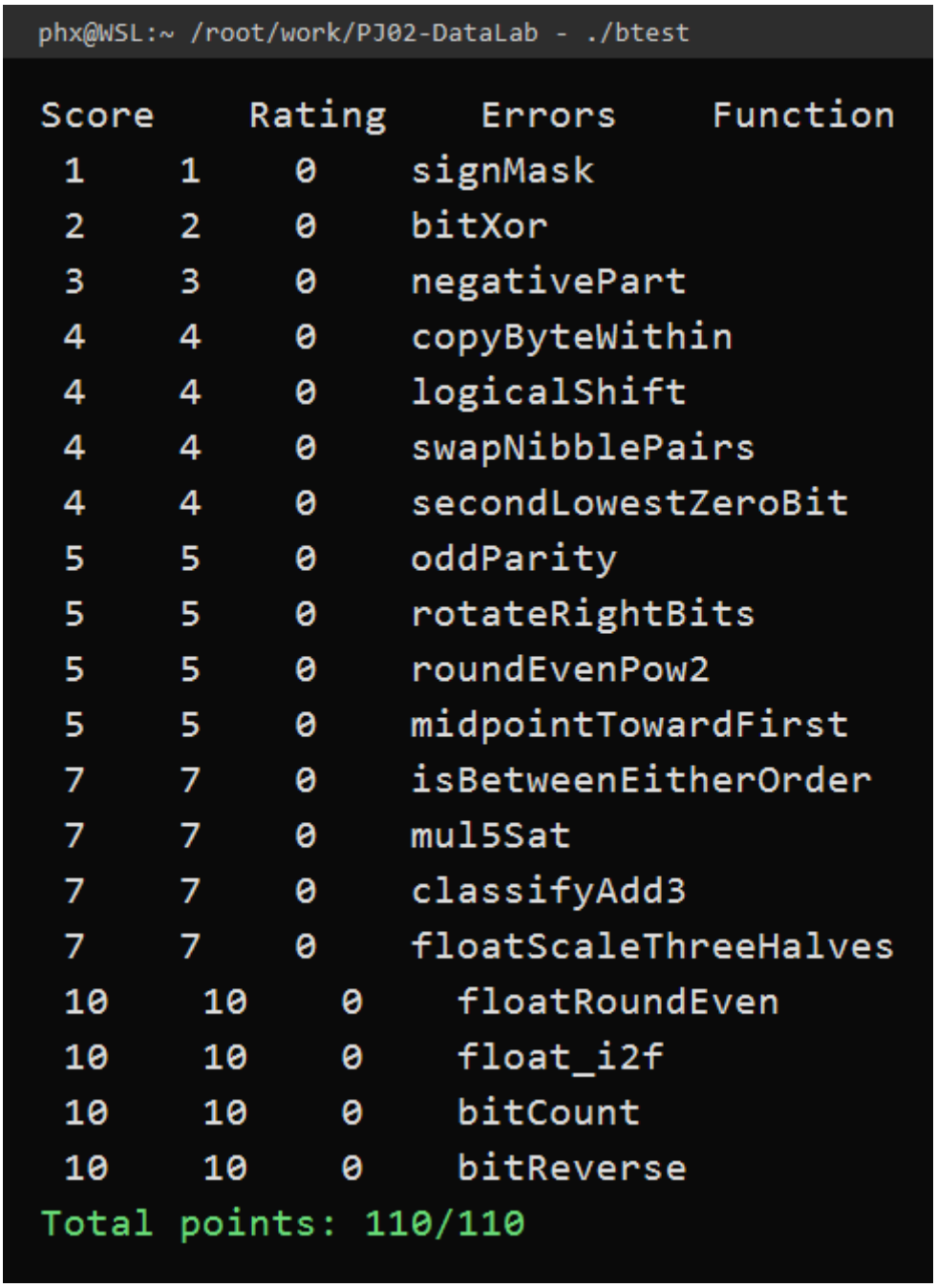
汇总如下（当前/上限）：

函数	操作数	上限	函数	操作数	上限
signMask	1	2	midpointTowardFirst	21	32
bitXor	7	8	isBetweenEitherOrder	39	48

函数	操作数	上限	函数	操作数	上限
negativePart	4	6	mul5Sat	7	30
copyByteWithin	10	12	classifyAdd3	14	52
logicalShift	6	20	floatScaleThreeHalves	45	60
swapNibblePairs	9	18	floatRoundEven	27	65
secondLowestZeroBit	8	8	float_i2f	26	40
oddParity	12	56	bitCount	28	40
rotateRightBits	12	16	bitReverse	32	34
roundEvenPow2	11	24			

二、功能测试截图

下图是在项目目录中执行 `./btest` 的输出。所有 19 题的 `Errors` 均为 `0`，最后一行为 `Total points: 110/110`。



除 `btest` 外，还执行了仓库自带的 `test.sh`（构建 + 规则检查 + 满分测试），结果输出 `all checks passed: 110/110`，退出码为 0。

三、各函数实现思路

整数题（仅允许 `! ~ & ^ | + << >>`，常量 0–255）

P1 `signMask` —— 返回最高位为 1 的掩码。
直接把 1 左移 31 位：`1 << 31`，即 `0x80000000`。

P2 `bitXor` —— 仅用 `~` 和 `&` 实现异或。
用德摩根律改写：`x ^ y = ~(~x & ~y) & ~(x & y)`。前半段覆盖「至少一个为 0」，后半段排除「同时为 1」的情况。

P3 `negativePart` —— `x<0` 时返回 `-x`，否则返回 0。
`x >> 31` 得到全 0（非负）或全 1（负数）的符号掩码，`~x + 1` 得到 `-x`，二者按位与即可：负数为 `-x`，非负数为 0。

P4 `copyByteWithin` —— 把第 `src` 字节复制到第 `dst` 字节。
先用 `(x >> (src<<3)) & 0xFF` 取出源字节，左移 `dst<<3` 位放好；同时用 `~(0xFF << (dst<<3))` 清空目标字节，最后按位或合并。

P5 `logicalShift` —— 逻辑右移（高位补 0）。
算术右移会补符号位，因此先用 `mask = ~(((1 << 31) >> n) << 1)` 生成「低 `32-n` 位为 1、高 `n` 位为 0」的掩码，再与 `x >> n` 按位与清除补入的符号位。`n=0` 时掩码为全 1，结果就是 `x`。

P6 `swapNibblePairs` —— 交换每个字节内的高、低半字节。
由 `0x0F` 逐级扩散构造掩码 `0x0F0F0F0F`，然后 `((x & mask) << 4) | ((x >> 4) & mask)`：低半字节上移 4 位、高半字节下移 4 位后合并。

P7 `secondLowestZeroBit` —— 标记 `x` 中「次低位的 0」所在位的掩码。
注意到「x 的 0 位」就是「`~x` 的 1 位」，故问题转化为求 `t = ~x` 的次低位 1。先用 `low = t & (~t + 1)` 取出最低位 1，再用 `rest = t ^ low` 清除它，最后 `rest & (~rest + 1)` 即次低位 1 的掩码；若不存在第二个 1（如 `x=-1` 或 `x=0x7FFFFFFF`）自然得到 0。恰好 8 个运算符。

P8 `oddParity` —— 1 的个数为偶数时返回 1，为奇数时返回 0。
先把 32 位用 XOR 折叠（`^ >> 16`、`^ >> 8`、`^ >> 4`、`^ >> 2`、`^ >> 1`），折叠后最低位即为奇偶校验位（奇数个 1 时为 1）。题目要求偶数为 1，故取反：`(x & 1) ^ 1`。

P9 `rotateRightBits` —— 循环右移，位数按 32 取模。
`n` 可能很大，先 `s = n & 31`。右半部分需要逻辑右移，用掩码 `~(((1 << 31) >> s) << 1)` 清除算术右移补入的符号位；左半部分移位量为 `(~s + 1) & 31`（等于 `(-s) mod 32`，从而避免使用被禁用的 `-`）。结果为 `((x >> s) & mask) | (x << s2)`。`s=0` 时右移量掩码为全 1、左移量为 0，结果仍为 `x`。

P10 `roundEvenPow2` —— 将非负 `x` 舍入到最近的 `2^n` 倍数，半程取偶。
令 `half = 1 << (n-1)`。先算商的奇偶 `lsb = (x >> n) & 1`，再把 `x + (half-1) + lsb` 右移 `n` 位得到商：`rem > half` 时进位、`rem < half` 时不进位、`rem == half` 时仅当商为奇数才进位（即「半程取偶」），用 `half-1+lsb` 一次性实现三种情况的进位判断。最后左移 `n` 位还原为目标倍数。

P11 `midpointTowardFirst` —— 无溢出的精确中点 `(x+y)/2`，半程偏向第一个参数 `x`。
先用位运算求下取整平均：`g = (x & y) + ((x ^ y) >> 1)`。仅在和为奇数（`tie = (x^y)&1`）时才需要 ±1，且当 `x >= y` 时取上整数（偏向 `x`）、否则取下整数。由于加减法可能溢出，这里用「符号位比较法」判 `x >= y`：符号相同看差值符号，符号不同直接由 `x` 的符号决定；`x==y` 不会出现在 `tie` 情况中。最终 `corr = tie & (x >= y)`，返回 `g + corr`。

P12 `isBetweenEitherOrder` —— 判断 `x` 是否落在以 `a`、`b` 为端点的闭区间内（两端点顺序任意）。
等价于 `(a<=x && x<=b) || (b<=x && x<=a)`。为避免减法溢出，实现了一个基于符号位的小于等于判定 `le(p,q)`：符号不同时结果为 `p<0`；符号相同时比较 `p + ~q` 的符号（同号相减不会溢出）。分别计算 `le(a,x)`、`le(x,b)`、`le(b,x)`、`le(x,a)` 后按上式组合，并对 0/1 做屏蔽。

P13 `mul5Sat` —— 计算 `5x`，溢出时饱和到 `INT_MAX` / `INT_MIN`。
关键是把溢出条件化为阈值比较。由 `0x33` 逐级构造 `t = 0x33333333`，`c = t >> 1 = 0x19999999`，于是正溢出阈值为 `A = c + 1 = 0x1999999A`（即 `⌊INT_MAX/5⌋+1`），负溢出阈值为 `B = ~c = -A`。用符号位比较判断 `x >= A`（正溢出）与 `x <= B`（负溢出），未溢出时结果为 `(x << 2) + x`，随后按掩码在 `INT_MAX`、`INT_MIN` 与普通结果间选择。

P14 `classifyAdd3` —— 判断精确和 `x+y+z` 相对 `[INT_MIN, INT_MAX]` 的大小（>INT_MAX 返回 1，<INT_MIN 返回 -1，否则 0）。
用「带符号进位」跟踪法：每次有符号加法若发生溢出，就根据两个加数符号判断真实和是多了还是少了一个 `2^32`（正溢出 `+1`、负溢出 `-1`），累加到一个高位变量 `hi` 中；经过两次加法（先 `x+y`，再 `+z`）后，`hi` 的符号与是否为 0 即决定分类结果。全程只用 32 位 `int`，未使用更宽类型。

浮点题（允许 `int/unsigned`、任意常量、算术/逻辑/比较与 `if`、`while`）

P15 `floatScaleThreeHalves` —— 计算单精度 `f * 3/2`，round-to-nearest-even，保留 ±0 符号，NaN 原样返回。
拆出符号位、阶码 `exp`、尾数 `frac`；`exp==0xFF`（NaN/Inf）与零值分别特判（零直接返回原值以保留符号）。把浮点还原为整数有效数 `P` 与指数 `E`：规格化时 `P = (frac | 0x800000) * 3`、`E = exp - 151`；非规格化时 `P = frac * 3`、`E = -150`（乘 1.5 已折进指数）。用循环求 `P` 的最高位 `n` 得结果的真实阶 `e = E + n`。若 `e >= -126` 为规格化结果，按 24 位尾数右移 `n-23` 位并用 round-half-even 舍入（含进位到 `2^24` 时阶加一的处理）；若 `e < -126` 为次正规结果，按 `2^-149` 的精度舍入，若舍入后达到 `2^23` 则返回最小规格化数 `2^-126`。阶溢出时返回无穷大。

P16 `floatRoundEven` —— 将单精度值舍入到最近整数（半程取偶），以浮点位模式返回；舍入为 0 时保留符号；NaN/Inf 原样返回。
`exp==0xFF` 原样返回；次正规数幅值 `< 2^-126 < 0.5`，直接舍为 0（保留符号）；`e0 = exp-127 >= 23` 说明已是整数，原样返回；`e0 < 0` 时幅值 `<1`：当 `e0==-1` 且 `frac≠0`（值在 `(0.5,1)`）舍入为 1.0，否则（含 `0.5` 半程取偶）舍为 0。其余情况先取含隐含 1 的 24 位尾数 `m`，按 `shift = 23-e0` 移出小数部分并用 round-half-even 得到整数 `v`，再用一个循环求 `v` 的最高位 `L`，把 `v` 转回浮点位模式（含指数 `L+127`）后叠加符号位。

P17 `float_i2f` —— 把 `int x` 转换为单精度浮点位模式。
`x==0` 返回 0；符号位取 `x & 0x80000000`，幅值 `abs = x`（`x<0` 时 `abs = ~x + 1`）。用循环找到 `abs` 的最高位 `e`。尾数：`e <= 23` 时直接左移对齐；`e > 23` 时右移 `e-23` 位并对被移出的位做 round-half-even，若进位溢出到 `2^24` 则尾数右移一位、阶加一。最后组合符号位、偏置阶 `e+127` 和 23 位尾数。

位统计/位反转题（整数规则）

P18 `bitCount` —— 统计二进制中 1 的个数。
采用「平行求 1」分治法：先用 `0x55555555` 把相邻两位相加、再用 `0x33333333` 每 2 位求和、用 `0x0F0F0F0F` 每 4 位求和，之后逐次 `x += x >> 8`、`x += x >> 16`，最后 `x & 0x3F` 即为 1 的个数。掩码均由 `0x55/0x33/0x0F` 经 `<< 8`、`<< 16` 扩散构造（满足常量 ≤255 的限制）。

P19 `bitReverse` —— 32 位整数的位反转。
用 5 级「交换」完成：先交换相邻 1 位（`0x55555555`）、再交换相邻 2 位（`0x33333333`）、相邻 4 位（`0x0F0F0F0F`）、相邻 8 位（`0x00FF00FF`），最后交换高、低 16 位（掩码 `0x0000FFFF`）。每级形如 `((x & M) << s) | ((x >> s) & M)`。所有掩码由 `0x55/0x33/0x0F/0xFF` 逐级构造；注意最终 16 位掩码必须用 `0xFF | (0xFF << 8)` 得到 `0xFFFF`（不能由 `0x00FF00FF >> 8` 得到，否则会得到 `0x0000FF00`）。

四、参考与引用

1. **Henry S. Warren Jr. *Hacker's Delight*, 2nd Edition**：平行位计数（popcount）、分治式位反转、无溢出求平均、round-half-to-even 舍入等经典位运算技巧的思想来源。

2. **IEEE Std 754-2019 / IEEE 754 单精度浮点数规范**：符号位、阶码偏置 127、隐含前导 1、非规格化数、NaN 与无穷大的位模式约定。
3. **运行环境**：WSL2 (Ubuntu 22.04.5 LTS)、GCC 11、`gcc-multilib` 与 `libc6-dev-i386`（32 位编译支持）。

附：全部结果一览

- `./check_ops.py bits.c`：`All 19 functions passed operator checks.`
- `./btest`：`Total points: 110/110`（19 题 `Errors` 均为 0）
- `bash test.sh`：`all checks passed: 110/110`，退出码 0